

Statewide Parcel Data ETL Pipeline

A modular, config-driven pipeline of ArcGIS Python Toolboxes and standalone Python scripts that turns raw county parcel data into a clean, classified, audited statewide dataset

The Problem

Turning raw parcel records for an entire state into clean, consistently classified, audited data is a heavy data engineering problem, especially at the scale of every parcel in Ohio. A handful of real obstacles show up immediately at this scale.

- Parcel data comes from many counties, each often digitized under different local conventions, land use coding standards, or geometry states (multipart polygons, near duplicate records, overlapping boundaries left over from data conflation).
- At statewide scale, millions of records, anything that isn't a well optimized spatial join or transform becomes painfully slow. A naive spatial join done with row-by-row cursor logic can take hours.
- Classification logic that assigns each parcel into a standardized category shouldn't live buried in code. As categories or business rules evolve, hardcoded mapping dictionaries become a liability, since every correction requires a code change and a redeploy.
- Data quality varies widely by county. Some counties have clean, complete land use coding, others have large amounts of non-standard or missing codes. A single statewide quality bar doesn't work, quality needs to be assessed county by county before deciding how much to trust that county's data.
- Nothing should silently ship bad data. Before a deliverable goes out, the numbers need to be checked against sanity bounds, with a human able to catch and approve exceptions rather than have them slip through unnoticed.

My Approach

I designed and built this as a modular pipeline of ArcGIS Python Toolboxes and standalone Python scripts, each tool owning one clearly scoped stage, so the whole pipeline can run end to end or be re-run at just the stage that needs it.

Configuration-driven business rules, not hardcoded dictionaries. Classification logic, county quality tiers, and validation thresholds all live in version-controlled CSV configuration files, synced into the geodatabase by a dedicated tool at the start of every session. A change to how a code should be classified is a one-line CSV edit, not a script change and redeploy.

Automated data acquisition with a safe test mode. Parcel data is pulled directly from a statewide parcel feature service. A built-in test mode lets an analyst validate the connection or inspect a schema against a small sample without ever touching the production dataset the rest of the pipeline reads from.

Multi-phase geometry and attribute cleaning. Raw parcels run through a configurable sequence: multipart polygons are exploded to single-part, geometric duplicates are identified and removed, and overlapping records above a configurable similarity threshold are resolved down to a single kept record using a defined tie-break rule rather than an arbitrary one. Every deleted record is preserved in a scratch database for audit and recovery, nothing is thrown away silently.

County-by-county data quality auditing before anything is trusted. Before classification, every county's data is scored on measurable metrics (coverage rate, share of non-standard codes, outlier parcel sizes), and the results are handed to an analyst who assigns a quality tier per county. The pipeline enforces this as a hard gate: classification will not run until every county has been reviewed and approved.

A parallelized spatial join for statewide scale. Assigning every parcel to its containing spatial zone is normally the most expensive step in a pipeline like this. I rebuilt it using OGR and GEOS with a process pool, splitting the zones across worker processes that each run their own spatial index lookups and write results to local staging files that get merged back at the end. That took the join from a multi-hour operation down to roughly half an hour.

Config-driven classification with a data-backed industrial split. Every parcel gets classified against the same configuration table used at sync time, so classification logic and configuration live in exactly one place. For industrial parcels specifically, whether a given code should be treated as heavy or light industrial is decided from actual employment data overlaid onto those parcels, rather than an assumption, with a documented fallback rule for the rare case with no data available.

A hard governance gate before final output. Before a final export, every result is checked against a set of validation rules (internal consistency checks, statewide ratio sanity bounds, comparisons against a prior baseline where one exists). Any error-level failure halts the export outright, and any warning requires an analyst to explicitly acknowledge it before the pipeline will proceed. Nothing ships without a human reviewing the exceptions.

Technical Highlights

- **Language/Platform:** Python, ArcGIS Pro (`arcpy`) Python Toolboxes, plus standalone Python scripts for performance-critical stages
- **Data acquisition:** REST feature service integration, with a safe test mode isolated from production data
- **Geometry processing:** Multipart-to-single-part explosion, geometric and overlap-based deduplication with configurable thresholds, geodesic area calculation

- **Performance engineering:** A parallelized spatial join using OGR/GEOS and a process pool, cutting a statewide spatial join from hours down to roughly 30 minutes
- **Data governance:** Config-driven business rules stored in version-controlled CSVs, county-level quality tiering, and analyst approval gates enforced in code before downstream stages can run
- **Classification:** A two-tier approach, direct configuration table lookups for most categories, plus a data-backed employment overlay for industrial subtype splits
- **Quality assurance:** Per-county and per-zone audit tables, a rules-driven validation stage with error and warning severities, and a hard gate before any final export
- **Output:** A structured, versioned CSV export accompanied by a full validation log, ready for downstream analysis

Why It Matters

A pipeline like this only earns trust if every number in the final export can be traced back to a defensible decision, whether that's a config-driven classification rule, an analyst-approved data quality tier, or a validation check that passed. Building governance gates directly into the pipeline, rather than relying on someone remembering to check things manually, means bad data gets caught and reviewed before it reaches a deliverable, and every classification decision stays auditable and easy to correct without touching code.